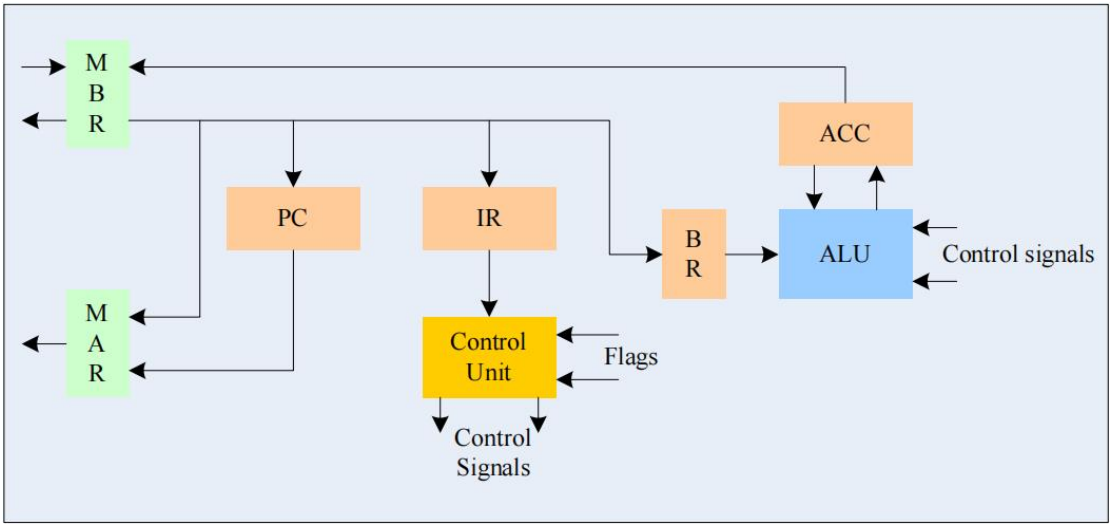
如何做一个 CPU？

一、 了解 CPU 的框架



理清楚这张图中的每个东西分别是做什么的，每一个部分都可以写一个文件，作为一个实体。实体的输入输出就是其在图中所连接的线。

拿 PC 举一个例子，可以看到它有一个输入一个输出，输入是来自于 MBR 的，输出是给 MAR 的。其他的输入还有例如系统时钟信号（clk）以及重置信号（rst），考虑到需要使用**微程序指令**来完成 CPU 的设计，因此输入还需要有控制信号。

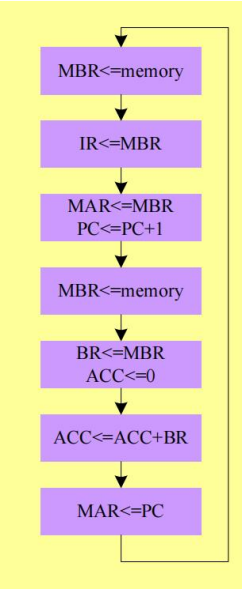
这里面比较难写的实体是 Control Unit 和 ALU，一个是用来输出控制信号的（Control Signals），一个是用来做各种计算的（加法、减法等），其中 Control Unit 需要详细了解微程序是如何实现的，这将会在下一部分中说明。

二、 复习微程序

所谓微程序，就是把一条条的机器指令给分割成若干小的基本操作，例如说：

LOAD X

这条指令可以看作是由以下一系列操作组成：

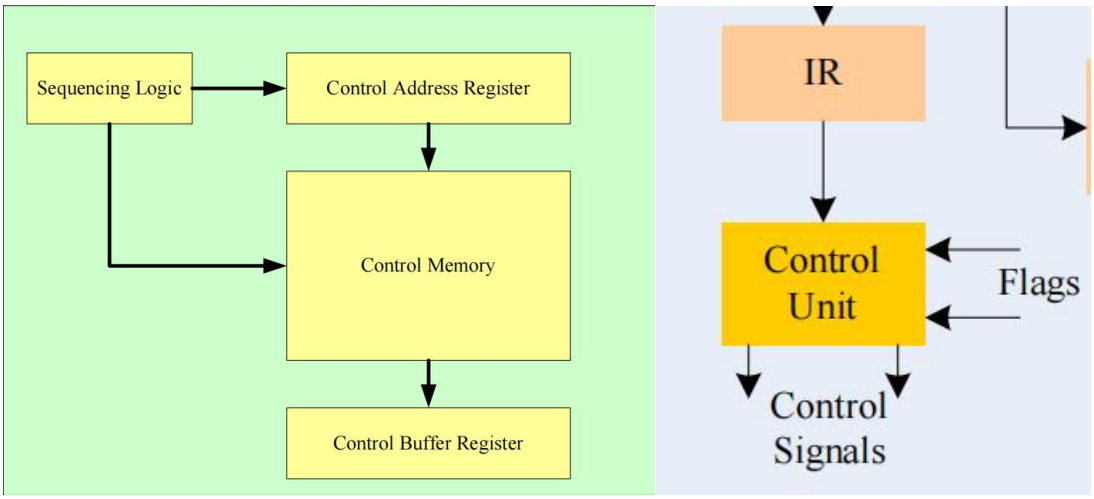


这里面的每一小步操作都可以看作是一个微操作，而微操作是需要通过控制信号来控制的，所谓控制信号，就是用来控制线路的通断，比如说这条 LOAD 指令的第一个微操作是将内存里的东西放入 MBR，那么 MBR 就需要有一个控制信号的输入信号，当该信号为 1 的时候，内存里的东西才能被成功放入 MBR，也就完成了控制线路通断。

更多具体的关于微程序的内容可以去看计组书。

三、 了解控制存储器（Control Unit）

控制存储器是 CPU 中比较难理解也是比较重要的一部分，其基本结构如上。其基本运作原理如下：



其中 Control Memory 是控制存储器，里面装载了 CPU 的全部微程序对应的微操作。Control Address Register 是控制地址寄存器（CAR），它告诉 CU 接下来要执行哪一条微操作，Control Buffer Register 是是控制缓冲寄存器（CBR），里面装载的其实就是产生的控制信号，它会输出到 CPU 的各个组件上，从而控制整个 CPU 的运作。因此，CU 的工作过程可以描述如下：

IR 输出 opcode（也就是指令集）给 CU。当 CU 接收到了 IR 输出的 opcode 后，它会做一个判断，根据所给的 opcode 解码出对应的微程序地址，然后把这个值赋给 CAR，让他告诉 CU 接下来该去 Control Memory 中找哪一段执行指令了。然后根据对应的微操作，会生成一段对应的控制信号，赋给 CBR，再输出出去。

特别需要注意的是，在我们的程序编写中 Sequencing Logic（定序逻辑）好像没有啥存在的意义。

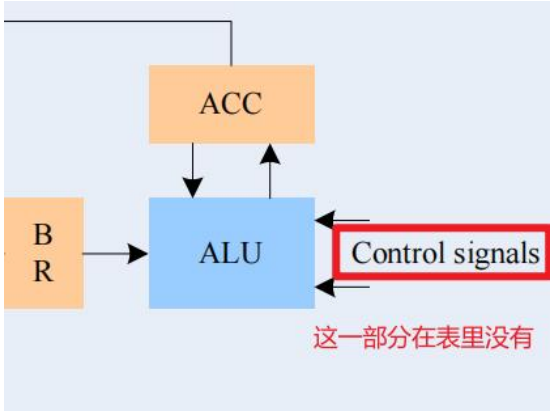
四、 写控制信号表

在了解了什么是微程序和控制存储器之后，我们需要写一个控制信号表。控制信号的含义已经在上面说过了，特定的控制信号是用来控制特定的线路通断，以控制数据在线路上的传送的，因此我们需要一张控制信号表来辅助我们后面编写控制存储表。

在老师所给的实验指导文档中已经给出了部分控制信号，这是他所定义的，当然你也可以重新来定义，这个问题不大，关键在于你需要将其补充完整。

Bit in Control Memory	*Micro-operation*	*Meaning*
C0	CAR<=CAR+1	Control Address Increment
C1	CAR<=***	Control Address Redirection, depends on the position of microinstruction
C2	CAR<=0	Reset Control Address to zero position
C3	MBR<=memory	Memory Content to MBR
C4	IR<=MBR[15..8]	Copy MBR[15..8] to IR for OPCODE
C5	MAR<=MBR[7..0]	Copy MBR[7..0] to MAR for address
C6	PC<=PC+1	Increment PC for indicating position
C7	BR<=MBR	Copy MBR data to BR for buffer to ALU
C8	ACC<=0	Reset ACC register to zero
C9	ACC<=ACC+BR	Add BR to ACC
C10	MAR<=PC	Copy PC value to MAR for next address
…	……	……

这里面的控制信号显然是不全的，可以看到少掉了对 ALU 的控制信号



对 ALU 的控制信号是什么呢？其实就是告诉 ALU，我接下来的这个运算是乘法还是加法还是减法还是与或非运算等等，ALU 接收到这个控制信号后，就知道自己要做什么运算了。

编写这部分的控制信号有两种方式：

1、直接法

你可以直接令 C11 控制加法运算，C12 控制减法运算……这样子的话，当对应的控制信号为 1 时，ALU 就知道自己该做什么运算了。

2、组合逻辑法

上面的直接法中，有多少种运算就需要多少个控制信号，但是我们可以通过一个组合逻辑，就可以实现少量的控制信号控制多种运算。

例如说我让 C11-C14 作为 ALU 的输入控制信号，当 C11-C14=0001 时代表加法运算 C11-C14=0010 代表减法运算……，这样子四个控制信号就可以实现十六种运算的控制。

五、 编写微程序

在这一部分中，我们需要给每一条指令都编写一段微程序，在实验文档中已经对 LOAD 指令做了一个基本的示例，如下所示：

Microprogram	*Control signals*
MBR<=memory, CAR<=CAR+1	C3, C0
IR<=MBR[15..8], CAR<=CAR+1	C4, C0
CAR<=*** (*** is determined by OPCODE)	C1
MAR<=MBR[7..0], PC<=PC+1, CAR<=CAR+1	C5, C6, C0
MBR<=memory, CAR<=CAR+1	C3, C0
BR<=MBR, ACC<=0, CAR<=CAR+1	C7, C8, C0
ACC<=ACC+BR, CAR<=CAR+1	C9, C0
MAR<=PC, CAR<=0	C10,C2

我们来解读一下这段微程序：

LOAD X 指令，就是把 X 中的内容装载到累加器 ACC 中

INSTRUCTION	OPCODE	COMMENTS
STORE X	00000001	ACC → [X]
LOAD X	00000010	[X] → ACC
ADD X	00000011	ACC + [X] → ACC
SUB X	00000100	ACC − [X] → ACC

我们将前三步连起来看，这是几乎所有指令的默认前三步：

第一步是将内存中的内容装载到 MBR 中，这其实是在读取 PC 的值，也即告诉 CPU 接下来要去执行哪个地址的指令了（搞不懂为什么是读取 PC 的值没关系，接着往下看）。然后 CAR 自增一，也就是告诉 CU，要执行下一条微操作了。

第二步，MBR 的高八位赋值到 IR 中，也就是将这条指令的操作码赋值给 IR，同时 CAR 自增一

第三步，CAR 根据 opcode 赋值，也即到指定的地址去执行接下来的其他微操作。

这前三步在 CU 的 Control Memory 中，地址应该分别是 0，1，2，是最前面的三条微操作，每次执行某条指令都是从他们开始，然后在第三步才会跳转到其他指定的地方去执行其他微操作（由 opcode 决定）

第四步，MBR 低八位赋值给 MAR，也就是把 LOAD X 这条指令中，X 的地址给到了 MAR，然后 PC 自增一，是为了方便下一条指令的执行，同时 CAR 自增一，为了方便下一条微操作的执行。

第五步，内存中的值赋值给 MBR，也就是把[X]赋值给了 MBR，这是因为 MAR 知道了 X 的地址，便会告诉内存，此时内存的地址便移动到了 X 所在处，从内存中取出来的内容也自然是 X 的值，也即[X]了。然后也是 CAR 自增一，就不多说了。

第六步，MBR 的值赋给 BR，就是说把[X]给到 BR 了，这是为了后面赋给 ACC 而准备的，同时 ACC 清零。

第七步，ACC+BR 的值赋值给 ACC，这其实要和上一步 ACC 清零连起来看，这两步的操作连起来，其实就是 BR 的值赋给了 ACC。这个时候你可能要问了，**为什么不直接把 BR 的值赋给 ACC 呢？**这是由我们所定义的控制信号表所决定的，返回去看控制信号表，发现并没有把 BR 赋值给 ACC 这一操作的对应控制信号，所以只能这么做（当然控制信号表你也可以重新定义啦）。

第八步，PC 的值赋给 MAR，CAR 清 0。这就告诉了 CPU 接下来一条指令的地址了，同时微程序又回到最开始的地址，重新开始执行一遍，但是读取的 opcode 可能就不同

了，因此实现了执行其他指令。

现在再返回去看第一步，现在你是否理解为什么是读取内存中的值就是读取 PC 的值了？因为每一条指令的微程序的最后一条微操作，都会把 PC 存到内存里，然后 CAR 置零，又回到了第一步。

在理解了 LOAD 这条指令的微程序后，你可以开始编写其他指令的微程序了，也像 LOAD 这样，写一个表格，把对应的微操作和控制信号写上去即可。

六、 写存储控制表

在完成了所有指令的微程序编写后，下一步便是把所有微程序整合写到一张表里面，并且把所有的微操作都指定一个地址（这里的地址是指 CU 中的，而不是内存中的），这里展示一下我们的部分存储控制表，以做参考，让你了解应该怎么写。

	C	D	E	F
1	指令	微操作	控制信号	控制存储
2		MBR<=memory, CAR<=CAR+1	C6, C15	b"01000000001000000"
3		IR<=MBR[15:8], CAR<=CAR+1	C5, C15	b"01000000000100000"
4		CAR<=***(***由OPCODE决定)	C16	b"10000000000000000"
5	STORE X	MAR<=MBR[7:0], PC<=PC+1, CAR<=CAR+1	C4, C9, C15	b"01000001000010000"
6		MBR<=ACC, CAR<=CAR+1	C2, C15	b"01000000000000100"
7		memory<=MBR, CAR<=CAR+1	C7, C15	b"01000000010000000"
8		MAR<=PC, CAR<=0	C1, C14	b"00100000000000010"
9	LOAD X	MAR<=MBR[7:0], PC<=PC+1, CAR<=CAR+1	C4, C9, C15	b"01000001000010000"
10		MBR<=memory, CAR<=CAR+1	C6, C15	b"01000000001000000"
11		BR<=MBR, ACC<=0, CAR<=CAR+1	C8, C10, C13, C15	b"01010010100000000"
12		ACC<=ACC+BR, CAR<=CAR+1	C10, C15	b"01000010000000000"
13		MAR<=PC, CAR<=0	C1, C14	b"00100000000000010"
14	ADD X	MAR<=MBR[7:0], PC<=PC+1, CAR<=CAR+1	C4, C9, C15	b"01000001000010000"
15		MBR<=memory, CAR<=CAR+1	C6, C15	b"01000000001000000"
16		BR<=MBR, CAR<=CAR+1	C8, C15	b"01000000100000000"
17		ACC<=ACC+BR, CAR<=CAR+1	C10, C15	b"01000010000000000"
18		MAR<=PC, CAR<=0	C1, C14	b"00100000000000010"
19		MAR<=MBR[7:0], PC<=PC+1, CAR<=CAR+1	C4, C9, C15	b"01000001000010000"
20		MBR<=memory, CAR<=CAR+1	C6, C15	b"01000000001000000"

可以看到，我在最后还加了一列控制存储，里面写的是一串二进制序列，对应的是我们的所有控制信号，从高到低分别代表了 C16-C0，哪个地方为 1 就代表那个控制信号有效了，这是为了方便我们后面编写 Control Unit 的程序，到时候就只需要复制粘贴就行了。

七、 写各个实体的程序

包括了 MAR、MBR、PC、IR、BR、ACC、ALU、CU，建议分工完成。在写完了这几个实体后，在写一个 CPU 的实体，将这些实体都例化进去，并完成相应线路的连接。

在 CPU 实体中需要定义以下几个输入输出：

首先时钟信号和重置信号没得说，而肯定要，其他的还需要有数据的输入输出线路，这是和内存相连接的，还要有读写控制以及地址信号，这也是服务于内存的。

读写控制信号其实也是由控制信号来决定，具体体现在 MAR、MBR 和 memory 的互动上。

最后再写一个顶层文件，例化 CPU 以及一个 RAM IP 核，该 IP 核就是 CPU 的内存，里面存储的是你要执行的程序的机器语言（就是 01 序列，当然也可以写成十六进制的）。

八、 编写 RAM 表

这部分其实就是在往内存里写东西，也就是写要执行的指令。例如说现在我要写一段 1 加到 100 的程序，那么首先我先用汇编语言把这段程序表示出来，然后再把它写成

对应的机器语言，最后写成一个 COE 文件，在设置 IP 核的时候读取该 COE 文件即可，以下展示我写的 RAM 表的一部分，便于你理解。

	A	B	C	D	E	F	G
1	地址	HEX	内容	对应指令	对应操作	说明	
2	0	0	02A0	LOAD A0	0 => ACC	将0赋值给sum，A0中的值为0，sum储存在A3中	1+2+...+100
3	1	1	01A3	STORE A3	ACC => sum		
4	2	2	02A2	LOAD A2	100 => ACC	将100赋值给temp，A2中的值为100，temp储存在A4中	
5	3	3	01A4	STORE A4	ACC => temp		
6	4	4	02A4	LOAD A4	temp => ACC	将temp值累加到sum中	
7	5	5	03A3	ADD A3	ACC + sum => ACC		
8	6	6	01A3	STORE A3	ACC => sum		
9	7	7	02A4	LOAD A4	temp => ACC	Temp中的值自减1，其中A1中存储着数值1	
10	8	8	04A1	SUB A1	ACC - 1 => ACC		
11	9	9	01A4	STORE A4	ACC => temp		
12	10	A	0504	JMPGER 04	04 => PC	循环	
13	11	B	02A5	LOAD A5	5 => ACC	5 * 6 = 30	测试乘法运算
14	12	C	08A6	MUL A6	ACC x 6 => MR, ACC		
15	13	D	01A9	STORE A9	ACC => result		
16	14	E	02A5	LOAD A5	5 => ACC	5 * -6 = -30	
17	15	F	08A8	MUL A8	ACC x -6 => MR, ACC		
18	16	10	01A9	STORE A9	ACC => result		
19	17	11	02A7	LOAD A7	-5 => ACC	-5 * -6 = 30	
20	18	12	08A8	MUL A8	ACC x -6 => MR, ACC		
21	19	13	01A9	STORE A9	ACC => result		
22	20	14	02A5	LOAD A5	5 => ACC	5(0101) and 6(0110) = 4(0100)	
23	21	15	0AA6	AND A6	ACC and 6 => ACC		

做一点小小的说明，例如第一条指令 LOAD A0，这里我定义了内存地址为 A0 的地方存储的是数字 0

1	地址	HEX	内容	对应指令	对应操作	说明
161	159	9F	0000			
162	160	A0	0000			0值
163	161	A1	0001			1值
164	162	A2	0064			100值
165	163	A3	0000			存储sum
166	164	A4	0000			存储temp
167	165	A5	0005			5值
168	166	A6	0006			6值
169	167	A7	FFFB			负5值
170	168	A8	FFFA			负6值
171	169	A9	0000			存储乘法result

这条指令的含义就是读取 0 值到 ACC 中，在 RAM 表中对应的内容就是 02A0，其中 02 代表的是 LOAD 指令的操作码，A0 代表的是读取数的地址。最后写进一个 COE 文件中，如下所示：

```
memory_initialization_radix=16;
memory_initialization_vector=02A0 01A3 02A2 01A4 02A4 03A3 01A3 02A4 04A1 01A4 0504
0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 00
```

最后就可以跑仿真来验证你的程序啦！

九、 最后

以上就是关于 CPU 设计的全部内容，关于后面上板测试，其实已经与 CPU 的设计不同了，所以此处不再多说，如果后面上板遇到问题可以再问。

整个设计的思路还是较为清晰的，也有助于你理解这个设计过程，而不是只是拿着别人的报告或者代码过来看一看就了事，那样子学习的效率不算太高。

最后，以上展示的内容，一部分是实验文档中本来就有的，另外一部分是由我个人完成的，因此也不太算侵犯到小组其他同学的权益，最后的最后，给出我自己写的 CU 的代码供你实在没思路时参考：

```
entity ControlUnit is
```

```
Port (
	clk,reset : in std_logic;
	IR_in : in std_logic_vector (7 downto 0);
	f0 : in std_logic;
	Control_signals : out std_logic_vector (16 downto 0)
);
end ControlUnit;

architecture Behavioral of ControlUnit is
signal CAR: std_logic_vector (7 downto 0) := "00000000";
signal CBR: std_logic_vector (16 downto 0);
signal counter1 : std_logic_vector (1 downto 0) := "00";
signal counter2 : std_logic_vector (0 downto 0) := "0";
begin
	Control_signals <= CBR;
	with CAR select
		CBR <= b"01000000001000000"	when x"00" ,
			b"01000000000100000"	when x"01" ,
			b"10000000000000000"	when x"02" ,
			b"01000001000010000"	when x"03" ,
			b"01000000000000100"	when x"04" ,
			b"01000000010000000"	when x"05" ,
			b"00100000000000010"	when x"06" ,
			b"01000001000010000"	when x"07" ,
			b"01000000001000000"	when x"08" ,
			b"01010010100000000"	when x"09" ,
			b"01000010000000000"	when x"0A" ,
			b"00100000000000010"	when x"0B" ,
			b"01000001000010000"	when x"0C" ,
			b"01000000001000000"	when x"0D" ,
			b"01000000100000000"	when x"0E" ,
			b"01000010000000000"	when x"0F" ,
			b"00100000000000010"	when x"10" ,
			b"01000001000010000"	when x"11" ,
			b"01000000001000000"	when x"12" ,
			b"01000000100000000"	when x"13" ,
			b"01000100000000000"	when x"14" ,
			b"00100000000000010"	when x"15" ,
			b"01000001000000000"	when x"16" ,
			b"00100000000000010"	when x"17" ,
			b"01000000000001000"	when x"18" ,
			b"00100000000000010"	when x"19" ,
			b"00100000000000010"	when x"1A" ,
			b"01000001000010000"	when x"1B" ,
			b"01000000001000000"	when x"1C" ,
			b"01000000100000000"	when x"1D" ,
```

```
                    b"01000110000000000"     when x"1E" ,
                    b"00100000000000010"     when x"1F" ,
                    b"01000001000010000"     when x"20" ,
                    b"01000000001000000"     when x"21" ,
                    b"01000000100000000"     when x"22" ,
                    b"01001000000000000"     when x"23" ,
                    b"00100000000000010"     when x"24" ,
                    b"01000001000010000"     when x"25" ,
                    b"01000000001000000"     when x"26" ,
                    b"01000000100000000"     when x"27" ,
                    b"01001010000000000"     when x"28" ,
                    b"00100000000000010"     when x"29" ,
                    b"01000001000010000"     when x"2A" ,
                    b"01000000001000000"     when x"2B" ,
                    b"01000000100000000"     when x"2C" ,
                    b"01001100000000000"     when x"2D" ,
                    b"00100000000000010"     when x"2E" ,
                    b"01000001000010000"     when x"2F" ,
                    b"01000000001000000"     when x"30" ,
                    b"01010010100000000"     when x"31" ,
                    b"01000010000000000"     when x"32" ,
                    b"01001110000000000"     when x"33" ,
                    b"00100000000000010"     when x"34" ,
                    b"01000001000010000"     when x"35" ,
                    b"01000000001000000"     when x"36" ,
                    b"01010010100000000"     when x"37" ,
                    b"01000010000000000"     when x"38" ,
                    b"01010000000000000"     when x"39" ,
                    b"00100000000000010"     when x"3A" ,
                    b"00000000000000000"     when others;

process(clk,reset)

begin
    if (clk'event and clk = '1') then
        if (reset = '1') then
            CAR <= "00000000";
        else
            if (CBR(6) = '1') then
                case counter1 is
                    when "10" =>
                        counter1 <= "00";
                        CAR <= CAR + 1;
                    when "01" =>
                        counter1 <= counter1 + 1;
                    when "00" =>
```

```
                        counter1 <= counter1 + 1;
                    when others => null;
                end case;
            elsif (CBR(7) = '1') then
                case counter2 is
                    when "1" =>
                        counter2 <= "0";
                        CAR <= CAR + 1;
                    when "0" =>
                        counter2 <= counter2 + 1;
                    when others => null;
                end case;
            elsif (CBR(14) = '1') then
                CAR <= "00000000";
            elsif (CBR(15) = '1') then
                CAR <= CAR + 1;
            elsif (CBR(16) = '1') then
                case IR_in is                                   --解码
                    when "00000001" => CAR <= x"03";            --STORE X
                    when "00000010" => CAR <= x"07";            --LOAD X
                    when "00000011" => CAR <= x"0C";            --ADD X
                    when "00000100" => CAR <= x"11";            --SUB X
                    when "00000101" =>                          --JMPGEZ X
                        if (f0 = '1') then
                            CAR <= x"18";                       --ACC >= 0
                        else
                            CAR <= x"16";                       --ACC < 0
                        end if;
                    when "00000110" => CAR <= x"18";            --JMP X
                    when "00000111" => CAR <= x"1A";            --HALT
                    when "00001000" => CAR <= x"1B";            --MPY X
                    when "00001010" => CAR <= x"20";            --AND X
                    when "00001011" => CAR <= x"25";            --OR X
                    when "00001100" => CAR <= x"2A";            --NOT X
                    when "00001101" => CAR <= x"2F";            --SHIFTR
                    when "00001110" => CAR <= x"35";            --SHIFTL
                    when others => null;
                end case;
            end if;
        end if;
    end if;
end process;
end Behavioral;
```